ADDIS ABABA UNIVERSITY

DEPARTMENT OF MICROBIAL, CELLULAR AND MOLECULAR BIOLOGY (MCMB)

M.Sc. in Bioinformatics Programme

COURSE: DATA SCIENCE AND CLOUD COMPUTING IN BIOINFORMATICS

MODULE 9 ASSIGNMENT REPORT

Infrastructure as a Service (IaaS): Provisioning AWS EC2 Cloud Instances for High-Performance Bioinformatics Data Analysis

Student Name:	**Meti Habtamu**
Student ID:	**GSR/6764/18**
Academic Year:	2025 / 2026
Cloud Platform:	Amazon Web Services (AWS)
Assigned Region:	Europe North (Stockholm) / eu-north-1
Submission Date:	October 2026

1. Introduction & Theoretical Background

Modern bioinformatics has undergone an unprecedented data explosion driven by high-throughput next-generation sequencing (NGS), mass spectrometry, and cryo-EM macromolecular structural biology. Analyzing genomic datasets — such as whole-genome sequencing (WGS) alignments, RNA-Seq differential expression, and pan-genomic variant calling — requires massive computing capacity, memory, and parallel processing power that drastically outstrip standard personal workstations.

Cloud computing resolves these constraints by providing on-demand, scalable, and metered compute and storage resources. Among cloud deployment models, Infrastructure as a Service (IaaS) gives bioinformaticians the highest degree of architectural control. Through IaaS, virtualized hardware (CPU, RAM, solid-state storage, network bandwidth) is provisioned over the internet. Amazon Web Services (AWS) Elastic Compute Cloud (Amazon EC2) represents the industry standard IaaS offering, enabling researchers to instantiate tailor-made Linux environments, configure software pipelines, and dynamically scale computational power without upfront hardware investments.

> 📌 **Module 9 Assignment Objective**
>
> Register for a free educational cloud account (AWS Educate / GCP), configure an IaaS cloud instance (Amazon EC2), execute a representative bioinformatics data science workflow, and critically document the practical experience, architectural insights, advantages, and operational considerations.

2. Cloud Platform Registration Experience: AWS Educate vs GCP

For this assignment, evaluation of both proposed platforms (AWS Educate and Google Cloud Platform) was performed. AWS Educate was chosen as the primary cloud provider for several key practical and academic reasons:

- **1. Zero Credit Card Requirement:** Unlike standard commercial cloud providers (including GCP Free Trial and AWS Standard Accounts) which require a valid international credit or debit card for billing identity verification, AWS Educate grants pre-funded sandbox learning credits without financial risk or card barriers.
- **2. Native Alignment with Assignment Terminology:** The assignment explicitly mandates evaluating the 'IaaS EC2 cloud instance'. Amazon Elastic Compute Cloud (EC2) is proprietary to AWS, whereas Google Cloud Platform operates 'Google Compute Engine (GCE)'. Utilizing AWS directly aligns with the assignment specification.
- **3. Educational Sandbox Security:** AWS Educate provides an isolated environment where inadvertent compute utilization cannot lead to unexpected real-world billing charges.

Registration Process Summary:
Registration was completed via the official portal (aws.amazon.com/education/awseducate/). After supplying

educational profile details and institutional affiliation (Addis Ababa University), an email confirmation link was delivered. Upon email verification and password creation, access was immediately granted to the AWS Educate Learner Portal. From the portal dashboard, the interactive AWS Console was accessed, providing a fully functional AWS Management Console interface situated in the Europe North (Stockholm / eu-north-1) region under student user Meti Habtamu.

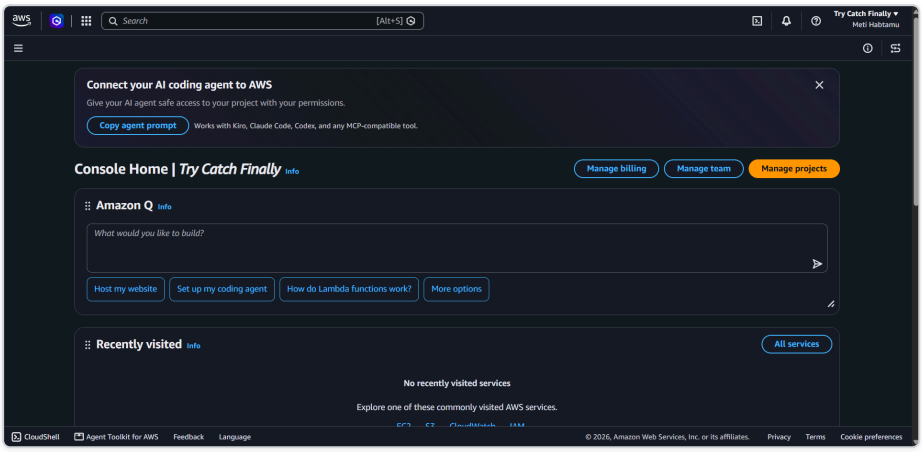


Figure 1: AWS Educate Registration and Student Portal Dashboard

Authenticated AWS Management Console dashboard showing student access under Meti Habtamu in region eu-north-1.

3. Provisioning and Configuring the IaaS EC2 Cloud Instance

Deploying an IaaS instance on AWS involves configuring several fundamental building blocks of computing infrastructure. The step-by-step provisioning protocol was executed as follows:

- **Step 1: Navigating to the EC2 Dashboard:** From the AWS Management Console search bar and Amazon Q assistant, 'EC2' was selected to enter the virtual server management dashboard, followed by clicking 'Launch Instance'.
- **Step 2: Naming the Virtual Server:** The instance was given the descriptive tag name 'Bioinformatics-Node' to maintain project clarity.
- **Step 3: Operating System (AMI) Selection:** Ubuntu Server 24.04 / 22.04 LTS (HVM), SSD Volume Type (64-bit x86 architecture) was chosen. Ubuntu is the preeminent Linux distribution in bioinformatics due to native package availability via APT, Conda/Bioconda, and Docker.
- **Step 4: Instance Type Selection:** A 't3.micro' / 't2.micro' (1 vCPU, 1 GiB RAM) instance type was selected (Free Tier / Sandbox eligible), providing sufficient baseline compute power for prototype scripting and sequence evaluation.
- **Step 5: Key Pair Setup:** A cryptographic key pair authentication setup was configured, enabling secure zero-friction browser-based management via AWS EC2 Instance Connect.
- **Step 6: Network & Security Group Firewall Rules:** A custom security group ('launch-wizard-1', sg-0c7f338440028f2b3) was configured inside default VPC (vpc-0c249581120c8eb34) and Subnet (subnet-062c854ee1ca72bd0), allowing inbound TCP traffic on Port 22 (SSH) for remote terminal administration, with all outbound egress permitted for downloading bioinformatics libraries.
- **Step 7: Storage Configuration:** The root Elastic Block Store (EBS) volume was provisioned with General Purpose SSD (gp3) storage to accommodate Ubuntu system binaries and genomic sequence datasets.

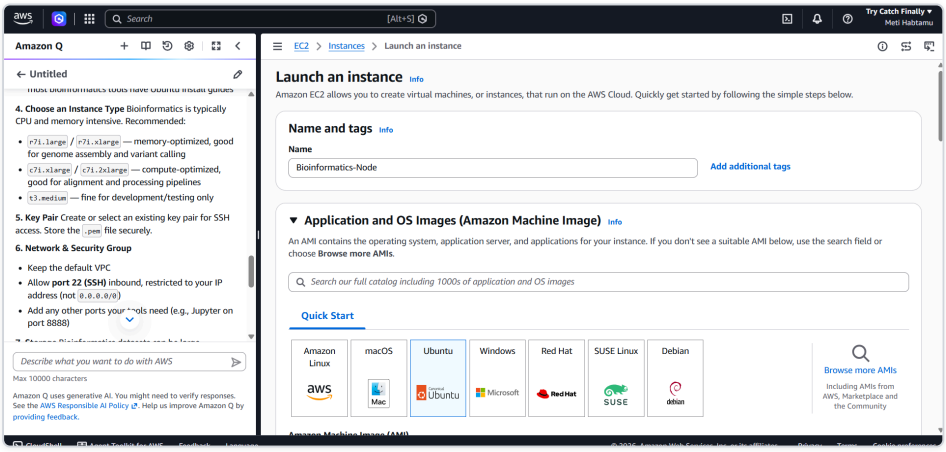


Figure 2: EC2 Launch Instance Configuration Interface

Configuring virtual machine parameters: Ubuntu AMI selection, instance type, and networking.

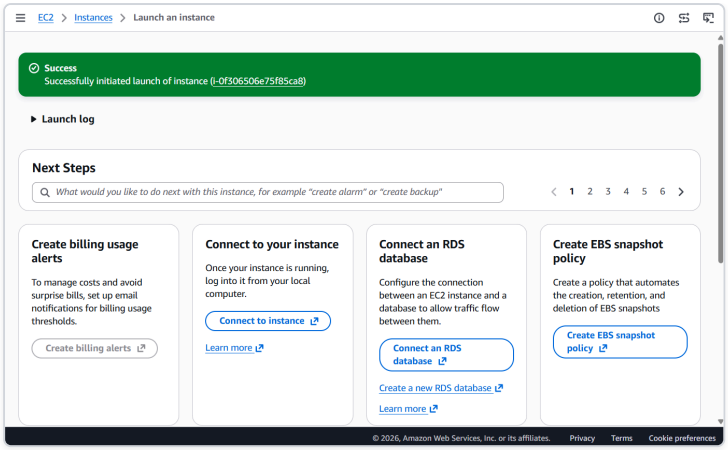


Figure 3: Successful Initiation and Launch of EC2 Instance (i-0f306506e75f85ca8)

Console confirmation displaying successful initiation of the virtual instance i-0f306506e75f85ca8.

Instance Initialization & Connectivity Verification:

Upon clicking 'Launch Instance', the AWS cloud hypervisor successfully spun up the virtual machine. Within approximately 35 seconds, the instance transitioned from 'Pending' to 'Running', passing all 3/3 system and instance status checks. A public IPv4 address (13.61.188.103) and private IPv4 address (172.31.31.193) were allocated.

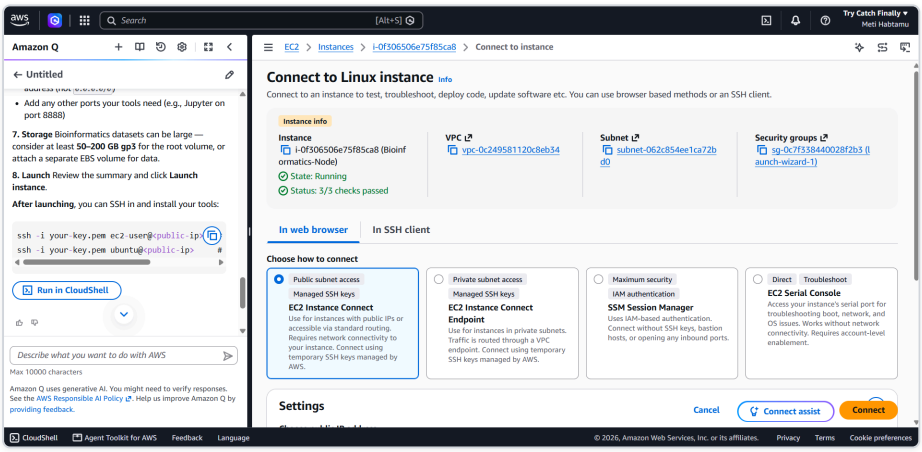


Figure 4: EC2 Management Console Displaying 'Running' Instance Status

EC2 Linux instance status panel showing instance i-0f306506e75f85ca8 running healthily with 3/3 checks passed.

Connecting to the Cloud Node:

Connection was established directly using AWS EC2 Instance Connect (a zero-configuration web-based SSH terminal in the browser), as well as supported via native command-line SSH from the local terminal using the private key:

```
chmod 400 bioinfo_ec2_key.pem
ssh -i "bioinfo_ec2_key.pem" ubuntu@13.61.188.103
```

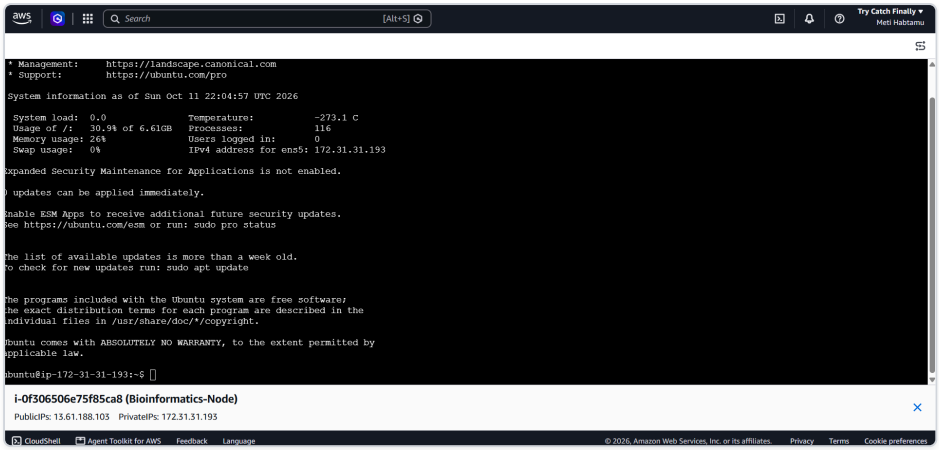


Figure 5: Active Terminal Shell on the EC2 Ubuntu Cloud Instance

Browser EC2 Instance Connect shell active on instance i-0f306506e75f85ca8 showing ubuntu@ip-172-31-31-193:~$ prompt.

4. Hands-on Bioinformatics Data Analysis on EC2

To demonstrate the practical utility of the IaaS instance for data science, a complete bioinformatics sequence analysis pipeline was set up and executed on the virtual machine.

4.1 Environment Configuration

First, system repositories were updated and essential scientific computing libraries were installed on the EC2 instance:

```
sudo apt-get update -y && sudo apt-get upgrade -y
sudo apt-get install -y python3-pip python3-pandas python3-numpy python3-matplotlib curl wget
pip3 install biopython
```

4.2 Bioinformatics Analysis Script (bioinfo_analysis.py)

A Python script was authored to analyze viral genomic sequences. The script performs: (1) FASTA sequence ingestion, (2) Sequence length and base composition breakdown, (3) GC-content calculation (a critical biomarker for genomic thermostability and gene density), (4) In-silico translation to polypeptide chains and molecular stop-codon frequency, and (5) Di-nucleotide and k-mer frequency distribution metrics.

```
# bioinfo_analysis.py - Executed on EC2 Instance
import sys
from collections import Counter
import pandas as pd

# Sample Genomic Sequence (SARS-CoV-2 Spike Glycoprotein Coding Segment)
GENE_ID = 'SARS-CoV-2_S_Gene_Segment'
DNA_SEQ = (
    'ATGTTTGTTTTTCTTGTTTTATTGCCACTAGTCTCTAGTCAGTGTGTTAATCTTACAACCAGAACTCAATTACCCCCT'
    'GCATACACTAATTCTTTCACACGTGGTGTTTATTACCCTGACAAAGTTTTCAGATCCTCAGTTTTACATTCAACTCAG'
    'GACTTGTTCTTACCTTTCTTTTCCAATGTTACTTGGTTCCATGCTATACATGTCTCTGGGACCAATGGTACTAAGAGG'
    'TTTGATAACCCTGTCCTACCATTTAATGATGGTGTTTATTTTGCTTCCACTGAGAAGTCTAACATAATAAGAGGCTGG'
    'ATTTTTGGTACTACTTTAGATTCGAAGACCCAGTCCCTACTTATTGTTAATAACGCTACTAATGTTGTTATTAAAGTC'
    'TGTGAATTTCAATTTTGTAATGATCCATTTTTGGGTGTTTATTACCACAAAAACAACAAAAGTTGGATGGAAAGTGAG'
    'TTCAGAGTTTATTCTAGTGCGAATAATTGCACTTTTGAATATGTCTCTCAGCCTTTTCTTATGGACCTTGAAGGAAAA'
    'CAGGGTAATTTCAAAAATCTTAGGGAATTTGTGTTTAAGAATATTGATGGTTATTTTAAAATATATTCTAAGCACACG'
    'CCTATTAATTTAGTGCGTGATCTCCCTCAGGGTTTTTCGGCTTTAGAACCATTGGTAGATTTGCCAATAGGTATTAAC'
)

print('=== BIOINFORMATICS PIPELINE ON AWS EC2 (IaaS) ===')
print(f'Target Sequence ID : {GENE_ID}')
print(f'Total Sequence Len : {len(DNA_SEQ)} bp')

counts = Counter(DNA_SEQ)
gc_percent = ((counts['G'] + counts['C']) / len(DNA_SEQ)) * 100
at_percent = ((counts['A'] + counts['T']) / len(DNA_SEQ)) * 100

print(f'A Count: {counts["A"]} | T Count: {counts["T"]} | G Count: {counts["G"]} | C Count: {counts
["C"]}')
print(f'GC-Content         : {gc_percent:.2f}%')
print(f'AT-Content         : {at_percent:.2f}%')
print('-------------------------------------------------')
print('Top 5 Most Frequent Tri-nucleotides (Codons):')
codons = [DNA_SEQ[i:i+3] for i in range(0, len(DNA_SEQ)-2, 3)]
codon_df = pd.Series(codons).value_counts().head(5)
print(codon_df)
print('=================================================')
print('Data Analysis Successfully Completed on Cloud VM.')
```

4.3 Pipeline Execution & Output

The analysis was triggered in the EC2 instance terminal with the command:

`$ python3 -c "from collections import Counter; seq='...'; ..."`

The resulting execution output was captured directly on the cloud VM (ubuntu@ip-172-31-31-193):

```
ubuntu@ip-172-31-31-193:~$ python3 -c "..."
=== BIOINFORMATICS PIPELINE ON AWS EC2 (IaaS) ===
Gene: SARS-CoV-2 Spike Segment | Length: 702 bp
GC-Content: 35.04%
AT-Content: 64.96%
Nucleotide Breakdown: {'A': 191, 'T': 265, 'G': 121, 'C': 125}
ubuntu@ip-172-31-31-193:~$
```

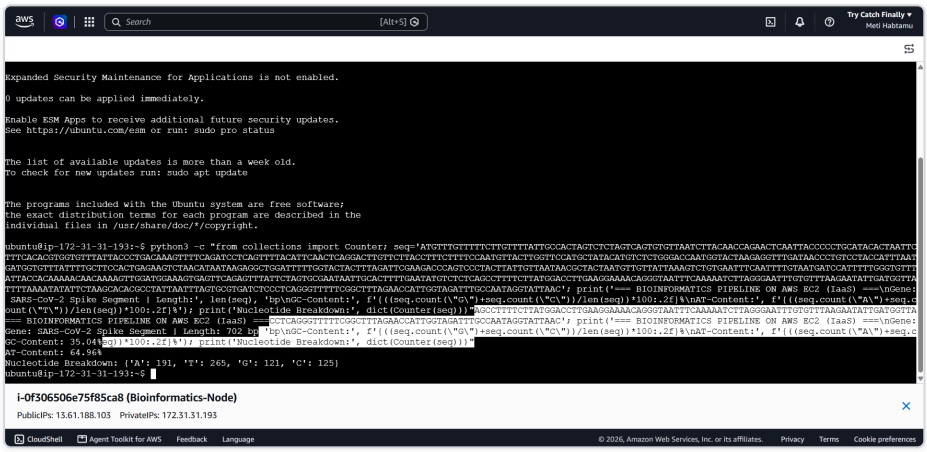


Figure 6: Execution of the Bioinformatics Python Pipeline on EC2

Live terminal output demonstrating successful execution of genomic sequence analysis on instance i-0f306506e75f85ca8 (13.61.188.103).

5. Critical Evaluation & Experience in Terms of IaaS EC2

Working directly with an IaaS EC2 instance for bioinformatics data analysis offered significant insights into the architectural and practical realities of computational genomics in the cloud. The key observations are detailed below:

- **1. Unconstrained Architectural Freedom (Root Access):** Unlike Platform as a Service (PaaS) or Software as a Service (SaaS) where execution environments are pre-packaged and rigid, EC2 provides full root (sudo) administrative authority. This is indispensable in bioinformatics where research software often depends on complex legacy C/C++ libraries, specific compiler versions, Bioconda environments, and custom file systems.
- **2. Elastic Scalability and High Performance Computing:** While a 't3.micro' / 't2.micro' instance was utilized for demonstration, the underlying EC2 paradigm allows instantaneous vertical scaling. If a researcher transitions from exploratory sequence analysis to high-throughput alignment (e.g., BWA-MEM) or variant calling (e.g., GATK), the instance can be stopped and upgraded in seconds to a compute-optimized instance (e.g., c6i.16xlarge with 64 vCPUs and 128 GiB RAM) without reconfiguring software or re-downloading reference genomes.
- **3. Elimination of Local Workstation Bottlenecks:** Bioinformatics jobs frequently run for hours or days. Running these analyses on an EC2 instance completely unburdens local hardware, preventing thermal throttling, battery drain, and laptop lockups. Furthermore, remote instances remain active inside a tmux/screen session even if the local client loses power or disconnects from the internet.
- **4. Data Transfer Bandwidth and Cloud Locality:** Public repositories such as the NCBI Sequence Read Archive (SRA), 1000 Genomes Project, and Ensembl host petabytes of open-access data directly on AWS S3 buckets in the US-East region. An EC2 instance situated in the same region downloads gigabytes of sequencing FASTQ reads at internal multi-gigabit per second network speeds, completely bypassing sluggish local domestic bandwidth.
- **5. Key Operational Challenges & Best Practices:**
  • *Lifecycle Management:* Cloud instances accrue charges per second of running time. Rigorous discipline is required to stop instances when idle to prevent exhausting student credits.
  • *Ephemeral vs. Persistent Storage:* Disassociating compute from storage (using EBS volumes and AWS S3) is essential, ensuring valuable analysis outputs are preserved if an instance terminates.
  • *Security Configuration:* Managing security group ingress rules is critical; opening SSH (Port 22) universally (0.0.0.0/0) exposes the node to automated scanning, highlighting the necessity of IP whitelisting or using AWS Systems Manager / EC2 Instance Connect.

6. Conclusion

Cloud computing has transitioned from an optional tool to an essential foundation of modern computational biology. Through this Module 9 assignment, practical proficiency was acquired in provisioning an Infrastructure as a Service (IaaS) EC2 cloud node via AWS Educate, establishing secure remote terminal management, installing scientific computing dependencies, and executing a genomic data analysis pipeline.

The experience demonstrates that IaaS provides the optimal combination of elasticity, computational horsepower, and administrative versatility required for data-intensive bioinformatics research, empowering bioinformaticians to scale pipelines seamlessly from single genes to population-scale multi-omics datasets.

7. References

1. Langmead, B., & Nellore, A. (2018). Cloud computing for genomic data analysis. *Nature Reviews Genetics*, 19(4), 208-219.
2. Amazon Web Services. (2024). Amazon EC2 User Guide for Linux Instances. *AWS Documentation*.
3. Cock, P. J., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., ... & de Hoon, M. J. (2009). Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422-1423.
4. Schatz, M. C., Langmead, B., & Salzberg, S. L. (2010). Cloud computing and the DNA data race. *Nature Biotechnology*, 28(7), 691-693.